

ARM Assembly Fundamentals

Introduction to ARM Cortex-M Microprocessors — Chapter 5

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1	<u>Why This Matters</u>
2	<u>How Much Assembly You Need</u>
3	<u>A First Program</u>
4	<u>The Load-Store Workflow</u>
5	<u>Load and Store Instructions</u>
6	<u>Loading Signed Numbers</u>
7	<u>Store Instructions</u>
8	<u>Registers in Practice</u>
9	<u>Constants and Addresses</u>
10	<u>Reading a Disassembly</u>
11	<u>Design Challenge</u>

1	Why This Matters	
2	How Much Assembly You Need	
3	A First Program	
4	The Load-Store Workflow	
5	Load and Store Instructions	
6	Loading Signed Numbers	
7	Store Instructions	
8	Registers in Practice	
9	Constants and Addresses	
10	Reading a Disassembly	
11	Design Challenge	

You have just written C code that reaches a hardware register. That code compiles into instructions. Sooner or later, you must read those instructions: when a debugger stops in a fault handler, when the compiler produces something you did not expect, when a timing loop must be exact, or when a startup file must run before any C code can execute. The next three chapters make the machine visible. Their goal is not to make you fluent in assembly. Instead, they let you look underneath any line of C code and see what the processor is really doing.

After completing this chapter, you should be able to:

1. Read a complete ARM assembly program and identify its directives, labels, and instructions.
2. Apply the load–process–store workflow to any operation on a memory variable.
3. Choose the correct load or store instruction for byte, halfword, and word data, and explain sign versus zero extension.
4. Trace a short instruction sequence register by register and predict its final state.
5. Distinguish low from high registers and explain why the choice affects code size.
6. Read a short disassembly and recover the C statement it came from.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

When you must read assembly

- A debugger halts in a **fault handler**
- The compiler produces something surprising
- A timing loop must be exact
- You are counting cycles in an ISR

When you must write it

- Startup code, before C can run
- ISR entry and exit in special cases
- Hand-tuned DSP inner loops

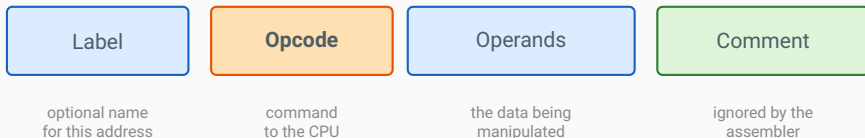
In short: **rarely**.

The goal of these three chapters

Not fluency, but **literacy**. You need enough skill to read a disassembly window and explain what the processor is doing, and why.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

The Shape of an Assembly Statement



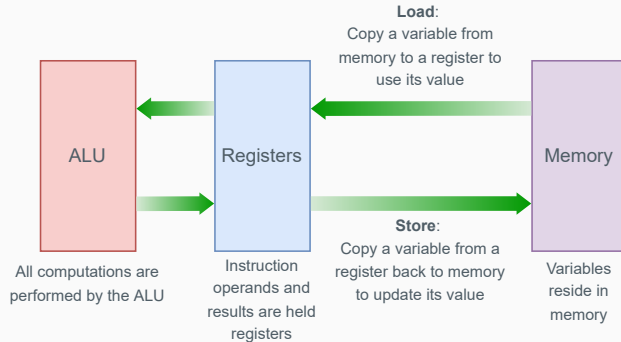
```
HERE    ADD    R1, R1, R0    ; add R0 to R1
```

The **first operand is the destination**. ADD R1, R1, R0 means “R1 becomes R1 plus R0.” Read it from right to left.

Directives (AREA, DCD, EQU, ENTRY, END) are commands to the **assembler**. They generate no machine code.

Instructions are commands to the **processor**.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge



Every operation on a memory variable follows the same three steps. No instruction can add two memory locations directly.

In C

```
counter = counter + 1;
```

What it must become

1. **Load** the value into a register
2. **Process** it in the ALU
3. **Store** it back

In assembly

```
LDR R1, =counter ; address of counter
LDR R0, [R1]      ; load its value
ADD R0, R0, #1    ; add one
STR R0, [R1]      ; store it back
```

Notice the two different LDRs: =counter loads the **address**.
[R1] loads the **value** at that address.

Why one C statement becomes four instructions

This is the load-store principle from Chapter 2. Here you see its real cost. This is not inefficiency. It is what keeps the clock fast and the timing predictable.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

Instruction	Size	Purpose
LDR / STR	word	Transfer 32 bits
LDRB / STRB	byte	Transfer 8 bits
LDRH / STRH	halfword	Transfer 16 bits
LDRSB	byte	Load byte and sign-extend
LDRSH	halfword	Load halfword and sign-extend

Naming convention

No suffix means a 32-bit **word**. B means **byte** (8 bits). H means **halfword** (16 bits).

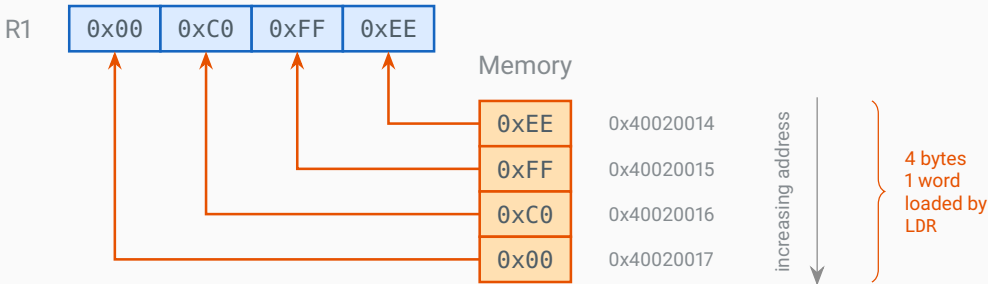
- An LDR-type instruction fetches data from memory and puts the value into the destination register, Rd.
- Syntax:

LDR Rd, [Rn, #offset]

- Rd: destination register, where the loaded value is stored
- Rn: base register, holding the memory address
- #offset: optional offset, added to the base register to compute the memory address

LDR: Loading a Word from Memory (Little-Endian)

LDR R0, [R1] ; load word

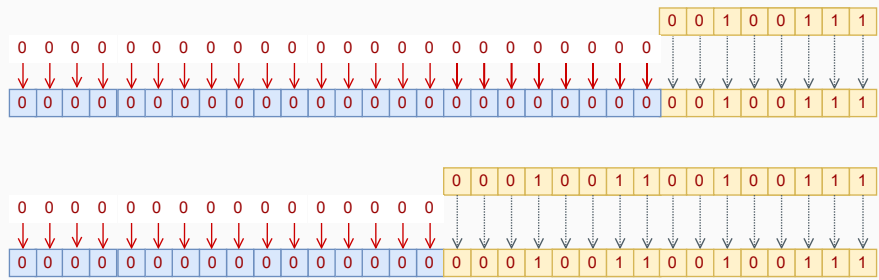


- Assume `R0 = 0x40020014`, and the **memory word** at `0x40020014` holds `0x00C0FFEE`.
- After `LDR R1, [R0]` runs, the destination register `R1` holds `0x00C0FFEE`.

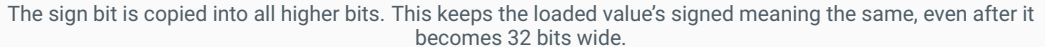
```
LDR    R0, [R1]    ; load 32 bits
LDRH   R0, [R1]    ; load 16 bits (H = halfword)
LDRB   R0, [R1]    ; load 8 bits (B = byte)
LDRSH  R0, [R1]    ; load 16 bits (SH = signed halfword)
LDRSB  R0, [R1]    ; load 8 bits (SB = signed byte)
```

A register is always 32 bits wide

When fewer than 32 bits are loaded, the processor must decide what fills the unused upper bits. This gives two important ideas: **zero extension** and **sign extension**.

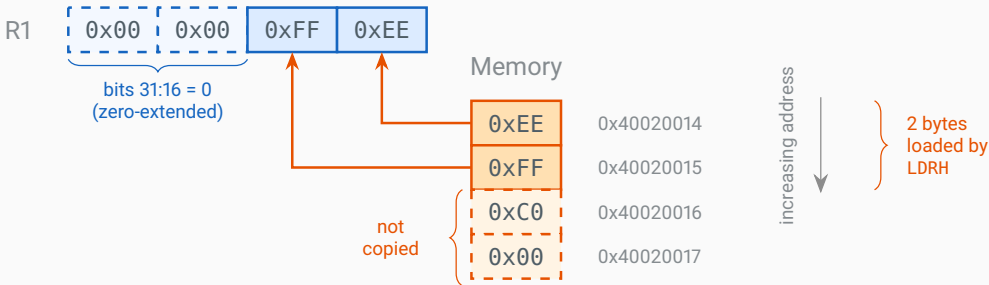


The loaded value is treated as **unsigned**. Every unused upper bit is set to zero.

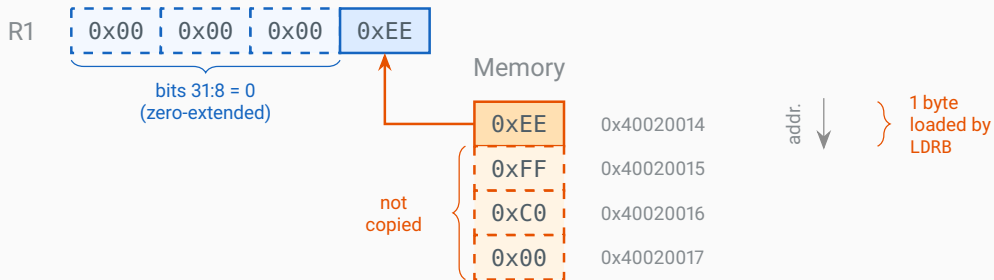


LDRH Instruction

LDRH R0, [R1] ; load halfword



- Assume R0 = 0x40020014, and the **memory halfword** at 0x40020014 holds 0xFFEE.
- After LDRH R1, [R0] runs, the lower halfword of R1 holds 0xFFEE, and the top halfword is **cleared**.



- Assume R0 = 0x40020014, and the **memory byte** at 0x40020014 holds 0xEE.
- After LDRB R1, [R0] runs, the lowest byte of R1 holds 0xEE, and the remaining bytes are **cleared**.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

The Same Bits Can Mean Different Numbers

- Most microprocessor work uses unsigned numbers.
- Sometimes, signed numbers are necessary

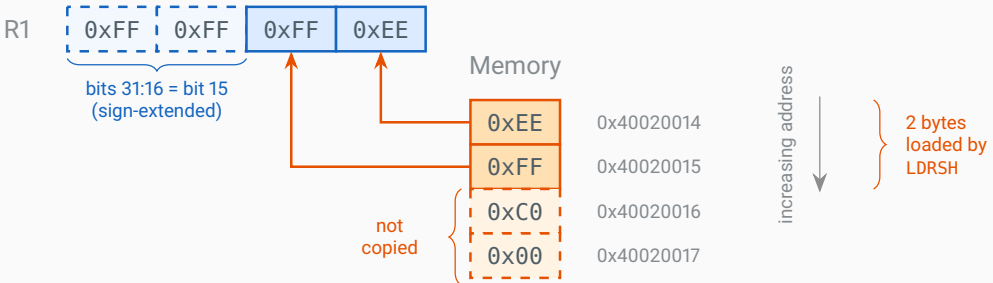
Byte in memory	LDRB	LDRSB
0x7F	0x0000007F = 127	0x0000007F = 127
0xFF	0x000000FF = 255	0xFFFFFFFF = -1
0x80	0x00000080 = 128	0xFFFFFFFF80 = -128

The instruction states your **intent**

Memory only stores bits. Choosing LDRB or LDRSB tells the processor whether those bits mean an unsigned value or a signed one.

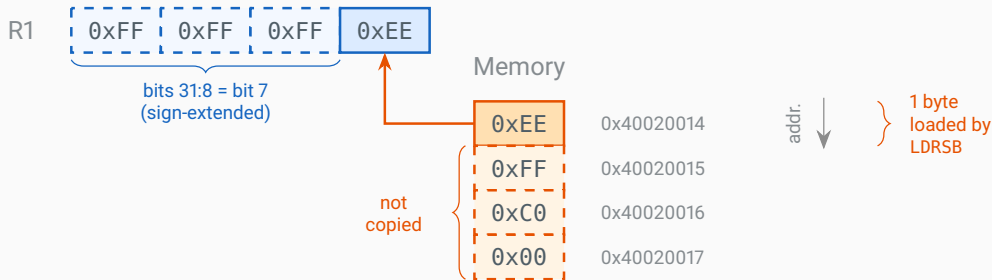
LDRSH Instruction

LDRSH R0, [R1] ; load signed halfword



- Assume R0 = 0x40020014, and the **memory halfword** at 0x40020014 holds 0xEEFF.
- After LDRH R1, [R0] runs, the lower halfword of R1 holds 0xFFEE, and the top halfword holds the sign bit of the loaded halfword.

LDRSB R0, [R1] ; load signed byte



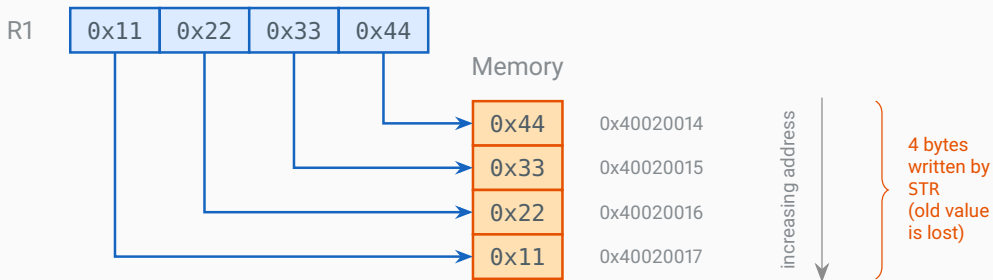
- Assume R0 = 0x40020014, and the **memory byte** at 0x40020014 holds 0xEE.
- After LDRSB R1, [R0] runs, the lowest byte of R1 holds 0xEE, and the remaining bytes hold the sign bit of the loaded byte.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

STR: Storing a Word to Memory

STR R0, [R1] ; store 32-bit word

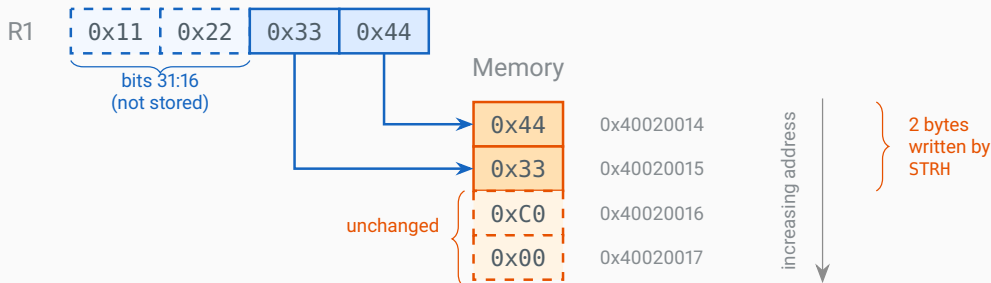
Before: R1 = 0x11223344, [0x40020014] = 0x00C0FFEE



STRH: Storing a Halfword to Memory

STRH R0, [R1]

Before: R1 = 0x11223344, [0x40020014] = 0x00C0FFEE

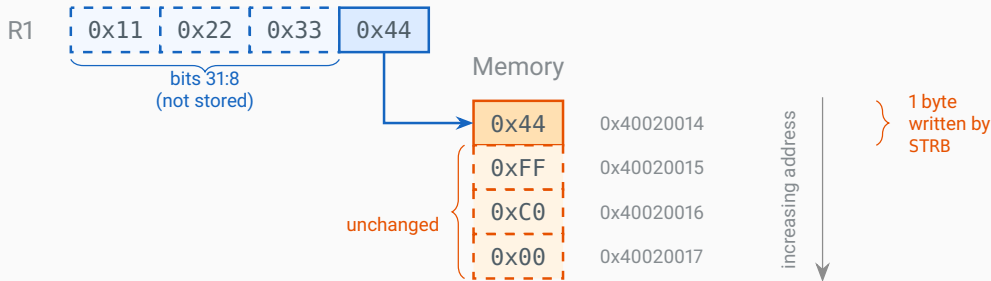


- Assume R0 = 0x40020014, R1 = 0x11223344.
- After STRH R1, [R0] runs, the **memory halfword** at 0x40020014 holds 0x4433.

STRB: Storing a Byte to Memory

STRB R0, [R1]

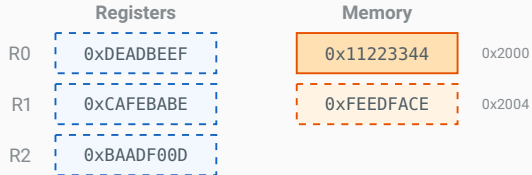
Before: R1 = 0x11223344, [0x40020014] = 0x00C0FFEE



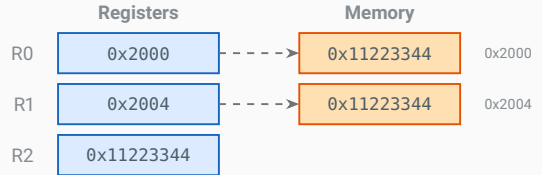
- After STRB R1, [R0] runs, the **memory byte** at 0x40020014 holds 0x44.

```
LDR R0, =0x2000  
LDR R1, =0x2004  
LDR R2, [R0]  
STR R2, [R1]
```

Before

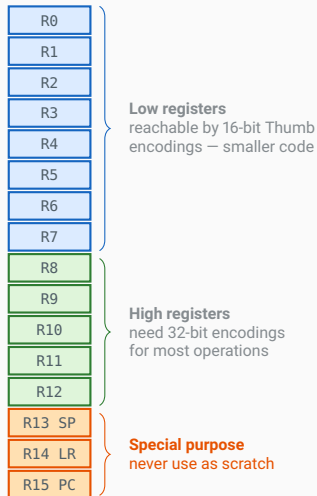


After



dashed arrow = address used for addressing

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge



Preferring R0–R7 is not just a style choice. It changes the size of the compiled program.

Seen in a disassembly

```
ADD R0, R1, R2 — 16-bit Thumb
```

```
ADD R8, R9, R10 — 32-bit Thumb-2
```

Same operation, but **double** the flash footprint.

R13/R14/R15 keep their special meanings. Never use them as scratch storage.

Special Registers You Will See

Three registers mean more than just “a place to put a number.” A disassembly window shows all three by name:

Register	Shown as	Role
R13	SP	Stack pointer — address of the last item pushed
R14	LR	Link register — BL writes the return address here
R15	PC	Program counter — writing it is a branch

The N/Z/C/V flags live in the **APSR**, part of the combined xPSR. This is often shown simply as PSR. Chapter 7 returns to these flags, once condition codes give them a use.

Prologue and epilogue

Mark start and end of a function

```

PUSH {LR}      ; save the return address
...            ; the function body
POP {PC}       ; return by loading PC directly

```

Two disassembly listings for the same C source always agree on *which* registers carry arguments and results. This rule is called the **AAPCS**.

Registers	Role
R0–R3	First four arguments in; R0 carries the return value out
R4–R11	<i>Callee-saved</i> — a function may use them, but must restore them

This is why `scale()`, traced next, reads and writes R0 with no register setup at all. The argument is already there.

Chapter 7 explains the full convention. Appendix B lists the complete standard.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

MOV and MVN: Register-to-Register Data Movement

- MOV copies a value into a register.
- The source can be another register or an encodable immediate.
- MVN performs the same operation after inverting every bit.

```
MOV R0, R1    ; copy R1 into R0
MOV R0, #25   ; load decimal 25
MVN R0, #0    ; R0 = 0xFFFFFFFF
MVN R0, R1    ; R0 = ~R1
```

What they cannot do

Neither instruction touches memory. Also, a single MOV cannot hold every possible 32-bit constant. So how do we load a large constant?

```
LDR R0, =0x40020014
```

- We often need constants that do not fit in a single MOV instruction.
- To avoid worrying about how it is encoded, we usually write LDR Rd, =value.
- This is an **assembler pseudo-instruction**. It means:

"Place this constant into the register."

Programmer versus assembler

You say *what* value you want. The assembler decides *how* to build it.

Option 1: Literal pool

```
LDR R0, [PC, #offset]
```

The constant sits in memory. The processor loads it using a PC-relative address.

Option 2: Build it directly

```
MOVW R0, #0x0014  
MOVT R0, #0x4002
```

The constant is built inside the register. No extra memory access is needed.

Key idea

LDR Rd, =value does not map to one specific machine instruction. It is just a request: put this constant into a register, however that is best done.

Constructing a 32-Bit Constant with MOVW/MOVT

```
MOVW R0, #0x0014 ; lower 16 bits
MOVT R0, #0x4002 ; upper 16 bits
```

R0 = 0x40020014

Why two instructions?

A Thumb-2 instruction cannot hold just any 32-bit constant.
 MOVW writes the lower half of the register. MOVT writes the upper half.

$\underbrace{0x4002}$	$\underbrace{0x0014}$
upper 16 bits	lower 16 bits

```
MOVW R3, #0x0014  
MOVT R3, #0x4002  
LDR R2, [R3]
```

How to read this sequence

Think of the first two instructions as one operation:

“Build the address 0x40020014 in R3.”

Once the address is in a register, the load can reach the memory-mapped peripheral register there.

In embedded programs, seeing a MOVW/MOVT pair usually means one address, or one large constant, is being built.

ARM can shift a register on its way *into* an instruction, at no extra cost — for example `ADD R0, R1, R2, LSL #2`. We will return to this feature in Chapter 6, where it becomes particularly useful for calculating addresses.

1	Why This Matters
2	How Much Assembly You Need
3	A First Program
4	The Load-Store Workflow
5	Load and Store Instructions
6	Loading Signed Numbers
7	Store Instructions
8	Registers in Practice
9	Constants and Addresses
10	Reading a Disassembly
11	Design Challenge

The listing

```
LDR R3, [R7, #4]  
LDR R2, [R7, #8]  
ADD R3, R3, R2  
STR R3, [R7, #12]
```

Reading it back

1. R7 is the **frame pointer**, so these are **local variables** at offsets 4, 8, and 12
2. Load two of them
3. Add them
4. Store the result in a third

The C was

```
c = a + b;
```

Three things to notice

The whole statement follows **load-process-store**. No S suffix appears, because nothing branches on the result. The offsets from R7 tell you these are locals on the **stack**, not global variables.

- The goal is **literacy**, not fluency: enough to read a disassembly window and say what the processor is doing.
- A statement has four fields: **label**, **opcode**, **operands**, **comment**. The first operand is the **destination**.
- Every operation on a memory variable follows **load–process–store**.
- LDR Rn, =label loads an **address**. LDR Rn, [Rm] loads a **value**.
- The suffix names the size: none = **word**, B = **byte**, H = **halfword**. LDRB/LDRH **zero-extend**; LDRSB/LDRSH **sign-extend**.
- **Low registers** R0–R7 are reachable by 16-bit instructions, so compilers prefer them.

Next: Chapter 6 — Memory, Arithmetic and Bit Manipulation

Next: how ARM calculates addresses efficiently, and the instructions that change individual bits.

1	Why This Matters	
2	How Much Assembly You Need	
3	A First Program	
4	The Load-Store Workflow	
5	Load and Store Instructions	
6	Loading Signed Numbers	
7	Store Instructions	
8	Registers in Practice	
9	Constants and Addresses	
10	Reading a Disassembly	
11	Design Challenge	

The brief

A colleague's driver reads a 12-bit ADC result with LDR (a full word), then masks off the top 20 bits in C. Another engineer suggests using LDRH instead, since the ADC data register is only 16 bits wide.

Compare the two approaches instruction by instruction: how many instructions does each need, and does either risk reading memory that does not belong to the ADC register? Then consider a signed sensor reading stored as a 16-bit two's-complement value. Explain which load instruction is required, and what goes wrong if the wrong one is used. State the general rule you would give a junior engineer for choosing between LDR, LDRH, and LDRSH.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 5